

Preprocessing and data cleaning

- **dataset1**: PSS-10, GAD-7, PHQ-9 totals and survey bands
- **dataset2**: DASS-21 (doubled subscale scores) and parsed demographics

```
import pandas as pd
from pathlib import Path

from utils import (
    print_dataset_summary,
    extract_number,
    age_int_to_bracket,
    cgpa_float_to_band,
    semester_int_to_academic_year,
    dass21_stress_score_to_level,
    dass21_anxiety_score_to_level,
    dass21_depression_score_to_level,
)
```

load raw tables

```
# assume notebook is run from proj_root/mhp/
ROOT = Path.cwd().parent
RAW_DATA = ROOT / "raw_data"
DATA = ROOT / "mhp" / "data"

dataset1 = pd.read_csv(
    RAW_DATA / "MHP Dataset of University Students_processed.csv"
)
dataset2 = pd.read_csv(
    RAW_DATA / "Dataset for Analyzing University Students Behaviour.csv",
    sep=";",
    encoding="utf-8",
)

for key, (title, frame) in {
    "dataset1": ("MHP Dataset of University Students", dataset1),
    "dataset2": (
        "Dataset for Analyzing University Students Behaviour",
        dataset2,
    ),
}.items():
    print_dataset_summary(key, title, frame)
```

constants shared by both datasets

```
AGE_ORDER = ["<18", "18-22", "23-26", "27-30", ">30"]
GENDER_LABELS = ["female", "male", "other"]

# department codes for dataset1
```

```

# dataset2 uses only "not_recorded" but categories must match
DEPARTMENT_CODES = sorted(
    {
        "bio_sciences",
        "business",
        "eng_cs",
        "eng_civil",
        "eng_eee",
        "eng_mech",
        "eng_other",
        "env_life_sciences",
        "law",
        "liberal_arts",
        "other",
        "pharmacy",
        "not_recorded",
    }
)

ACADEMIC_YEAR_ORDER = ["unknown", "first", "second", "third", "fourth", "other"]
CGPA_ORDER = [
    "other",
    "<2.50",
    "2.50-2.99",
    "3.00-3.39",
    "3.40-3.79",
    "3.80-4.00",
]

SEVERITY_ORDER = [
    "none",
    "minimal",
    "mild",
    "moderate",
    "moderately_severe",
    "severe",
]

GENDER_FROM_RAW = {
    "Female": "female",
    "Male": "male",
    "Prefer not to say": "other",
}

OUTPUT_COLUMNS = [
    "age",
    "gender",
    "department",
    "academic_year",
    "cgpa",
    "stress_level",
    "stress_z_score",
    "anxiety_level",
    "anxiety_z_score",
    "depression_level",
]

```

```
    "depression_z_score",  
  ]
```

dataset1: column renames, value maps, output schema

```
DATASET1_COLUMN_MAP = {  
    "Age": "age",  
    "Gender": "gender",  
    "Department": "department",  
    "Academic_Year": "academic_year",  
    "Current_CGPA": "cgpa",  
    "Stress Value": "pss10_stress_score",  
    "Stress Label": "stress_level",  
    "Anxiety Value": "gad7_anxiety_score",  
    "Anxiety Label": "anxiety_level",  
    "Depression Value": "phq9_depression_score",  
    "Depression Label": "depression_level",  
}  
  
DATASET1_VALUE_MAPS = {  
    "age": {  
        "Below 18": "<18",  
        "18-22": "18-22",  
        "23-26": "23-26",  
        "27-30": "27-30",  
        "Above 30": ">30",  
    },  
    "department": {  
        "Biological Sciences": "bio_sciences",  
        "Business and Entrepreneurship Studies": "business",  
        "Engineering - CS / CSE / CSC / Similar to CS": "eng_cs",  
        "Engineering - Civil Engineering / Similar to CE": "eng_civil",  
        "Engineering - EEE/ ECE / Similar to EEE": "eng_eee",  
        "Engineering - Mechanical Engineering / Similar to ME": "eng_mech",  
        "Engineering - Other": "eng_other",  
        "Environmental and Life Sciences": "env_life_sciences",  
        "Law and Human Rights": "law",  
        "Liberal Arts and Social Sciences": "liberal_arts",  
        "Other": "other",  
        "Pharmacy and Public Health": "pharmacy",  
    },  
    "academic_year": {  
        "First Year or Equivalent": "first",  
        "Second Year or Equivalent": "second",  
        "Third Year or Equivalent": "third",  
        "Fourth Year or Equivalent": "fourth",  
        "Other": "other",  
    },  
    "cgpa": {  
        "Other": "other",  
        "Below 2.50": "<2.50",  
        "2.50 - 2.99": "2.50-2.99",  
        "3.00 - 3.39": "3.00-3.39",  
        "3.40 - 3.79": "3.40-3.79",  
        "3.80 - 4.00": "3.80-4.00",  
    },  
}
```

```

    },
    "stress_level": {
        "Low Stress": "minimal",
        "Moderate Stress": "moderate",
        "High Perceived Stress": "severe",
    },
    "anxiety_level": {
        "Minimal Anxiety": "minimal",
        "Mild Anxiety": "mild",
        "Moderate Anxiety": "moderate",
        "Severe Anxiety": "severe",
    },
    "depression_level": {
        "No Depression": "none",
        "Minimal Depression": "minimal",
        "Mild Depression": "mild",
        "Moderate Depression": "moderate",
        "Moderately Severe Depression": "moderately_severe",
        "Severe Depression": "severe",
    },
}

DATASET1_OUTPUT_COLUMNS = [
    "age",
    "gender",
    "department",
    "academic_year",
    "cgpa",
    "pss10_stress_score",
    "stress_level",
    "stress_z_score",
    "gad7_anxiety_score",
    "anxiety_level",
    "anxiety_z_score",
    "phq9_depression_score",
    "depression_level",
    "depression_z_score",
]

```

dataset1: build dataset1_clean

```

dataset1_clean = dataset1[list(DATASET1_COLUMN_MAP.keys())].copy()
dataset1_clean = dataset1_clean.rename(columns=DATASET1_COLUMN_MAP)

dataset1_clean["gender"] = dataset1_clean["gender"].map(GENDER_FROM_RAW)

for col, mapping in DATASET1_VALUE_MAPS.items():
    dataset1_clean[col] = dataset1_clean[col].map(mapping)

for col, cats in (
    ("age", AGE_ORDER),
    ("academic_year", ACADEMIC_YEAR_ORDER),
    ("cgpa", CGPA_ORDER),
    ("stress_level", SEVERITY_ORDER),

```

```

    ("anxiety_level", SEVERITY_ORDER),
    ("depression_level", SEVERITY_ORDER),
):
    dataset1_clean[col] = pd.Categorical(
        dataset1_clean[col], categories=cats, ordered=True
    )

dataset1_clean["gender"] = pd.Categorical(
    dataset1_clean["gender"], GENDER_LABELS, ordered=False
)
dataset1_clean["department"] = pd.Categorical(
    dataset1_clean["department"], DEPARTMENT_CODES, ordered=False
)

# calculate z-scores
dataset1_clean["stress_z_score"] = (
    dataset1_clean["pss10_stress_score"]
    - dataset1_clean["pss10_stress_score"].mean()
) / dataset1_clean["pss10_stress_score"].std()
dataset1_clean["anxiety_z_score"] = (
    dataset1_clean["gad7_anxiety_score"]
    - dataset1_clean["gad7_anxiety_score"].mean()
) / dataset1_clean["gad7_anxiety_score"].std()
dataset1_clean["depression_z_score"] = (
    dataset1_clean["phq9_depression_score"]
    - dataset1_clean["phq9_depression_score"].mean()
) / dataset1_clean["phq9_depression_score"].std()

# reorder columns
dataset1_clean = dataset1_clean[DATASET1_OUTPUT_COLUMNS]

dataset1_clean.info()
dataset1_clean.head()

```

DASS-21 (Depression, Anxiety, and Stress Scale)

The [DASS-21](#) measures Depression, Anxiety, and Stress through 21 items (7 questions per category). Answers are scored 0 to 3.

The rating scale is as follows:

- **0:** Did not apply to me at all — *NEVER*
- **1:** Applied to me to some degree, or some of the time — *SOMETIMES*
- **2:** Applied to me to a considerable degree, or a good part of time — *OFTEN*
- **3:** Applied to me very much, or most of the time — *ALMOST ALWAYS*

Depression (7 items): Measures lack of interest, hopelessness, and dysphoria.

- I couldn't seem to experience any positive feelings at all
- I found it difficult to work up the initiative to do things
- I felt that I had nothing to look forward to
- I felt down-hearted and blue

- I was unable to become enthusiastic about anything
- I felt I wasn't worth much as a person
- I felt that life was meaningless

Anxiety (7 items): Measures physical arousal and situational panic.

- I was aware of the dryness of my mouth
- I experienced breathing difficulty (e.g. excessively rapid breathing, breathlessness in the absence of physical exertion)
- I experienced trembling (e.g. in the hands)
- I was worried about situations in which I might panic and make a fool of myself
- I felt I was close to panic
- I was aware of the action of my heart in the absence of physical exertion (e.g. sense of heart rate increase, heart missing a beat)
- I felt scared without any good reason

Stress (7 items): Measures tension, agitation, and difficulty relaxing.

- I found it hard to wind down
- I tended to over-react to situations
- I felt that I was using a lot of nervous energy
- I found myself getting agitated
- I found it difficult to relax
- I was intolerant of anything that kept me from getting on with what I was doing
- I felt that I was rather touchy

Severity Level	Depression Score	Anxiety Score	Stress Score
Normal	0 - 9	0 - 7	0 - 14
Mild	10 - 13	8 - 9	15 - 18
Moderate	14 - 20	10 - 14	19 - 25
Severe	21 - 27	15 - 19	26 - 33
Extremely Severe	28+	20+	34+

dataset2: column renames, constants, value maps, output schema

```
# source .csv column names
# DASS items = last 21 columns
DATASET2_COLUMN_MAP = {
    "Semester( number like: 1st, 2nd...)": "semester_raw",
    "Age( Years: 18, 19....)": "age_raw",
    "Gender": "gender",
    "Academic performance(CGPA like: 3.5.....)": "cgpa_raw",
}

# DASS-21 answers to 0-3 (must match .csv wording)
```

```

DATASET2_DASS_ANSWER_MAP = {
    "Did not apply to me at all": 0,
    "Applied to me to some degree, or some of the time": 1,
    "Applied to me to a considerable degree or a good part of time": 2,
    "Applied to me very much or most of the time": 3,
}

# standard DASS-21 ordering
DATASET2_DASS_STRESS_INDICES = [1, 6, 8, 11, 12, 14, 18]
DATASET2_DASS_ANXIETY_INDICES = [2, 4, 7, 9, 15, 19, 20]
DATASET2_DASS_DEPRESSION_INDICES = [3, 5, 10, 13, 16, 17, 21]

DATASET2_OUTPUT_COLUMNS = [
    "age_parsed",
    "age",
    "gender",
    "department",
    "semester_parsed",
    "academic_year",
    "cgpa_parsed",
    "cgpa",
    "dass21_stress_score",
    "stress_level",
    "stress_z_score",
    "dass21_anxiety_score",
    "anxiety_level",
    "anxiety_z_score",
    "dass21_depression_score",
    "depression_level",
    "depression_z_score",
]

```

dataset2: build dataset2_intermediate and dataset2_clean

```

dass21_src_cols = list(dataset2.columns[-21:])
DASS_COLS_MAP = {
    row: f"dass_{i:02d}" for i, row in enumerate(dass21_src_cols, start=1)
}
dataset2_wide = dataset2[
    list(DATASET2_COLUMN_MAP.keys()) + dass21_src_cols
].copy()
dataset2_wide = dataset2_wide.rename(
    columns={**DATASET2_COLUMN_MAP, **DASS_COLS_MAP}
)

print("DASS mapping:")
for k, v in DASS_COLS_MAP.items():
    print(f"- {v}: {k.strip()}")

```

parse age

```

dataset2_wide["age_parsed"] = (
    dataset2_wide["age_raw"]

```

```

        .apply(lambda x: round(extract_number(x)))
        .astype("Int64")
    )

print("mapping: raw age -> parsed age")
mapping_df = (
    dataset2_wide[["age_raw", "age_parsed"]]
    .drop_duplicates()
    .sort_values("age_parsed")
)
display(mapping_df)

```

parse semester

```

# manually defined mapping after inspection of semester values
_WORD_TO_SEMESTER = {
    "second": 2,
    "last": 9, # map to "other"
}

def parse_semester(value: str) -> int | None:
    n = extract_number(value)
    if n is not None:
        return int(n)
    clean = str(value).strip().lower()
    for word, num in _WORD_TO_SEMESTER.items():
        if word in clean:
            return num
    return None

dataset2_wide["semester_parsed"] = (
    dataset2_wide["semester_raw"].apply(parse_semester).astype("Int64")
)

print("mapping: raw semester -> parsed semester")
mapping_df = (
    dataset2_wide[["semester_raw", "semester_parsed"]]
    .drop_duplicates()
    .sort_values("semester_parsed")
)
display(mapping_df)

```

```

# manual fixes (invalid or year-like semester strings)
BAD_SEM_IDX = [129, 200, 307, 340]
dataset2_wide.loc[BAD_SEM_IDX, "semester_parsed"] = pd.NA

```

parse cgpa

```

dataset2_wide["cgpa_parsed"] = (
    dataset2_wide["cgpa_raw"].apply(extract_number).astype("Float64")
)

```



```

print("mapping: raw cgpa -> parsed cgpa")
mapping_df = (
    dataset2_wide[["cgpa_raw", "cgpa_parsed"]]
    .drop_duplicates()
    .sort_values("cgpa_parsed")
)
display(mapping_df)

```

parse DASS scores

```

dass_cols = [c for c in dataset2_wide.columns if c.startswith("dass_")]
for c in dass_cols:
    dataset2_wide[c] = (
        dataset2_wide[c].map(DATASET2_DASS_ANSWER_MAP).astype("Int64")
    )

stress_cols = [f"dass_{i:02d}" for i in DATASET2_DASS_STRESS_INDICES]
anxiety_cols = [f"dass_{i:02d}" for i in DATASET2_DASS_ANXIETY_INDICES]
depression_cols = [f"dass_{i:02d}" for i in DATASET2_DASS_DEPRESSION_INDICES]

dataset2_wide["dass21_stress_score"] = (
    dataset2_wide[stress_cols].sum(axis=1, min_count=7) * 2
)
dataset2_wide["dass21_anxiety_score"] = (
    dataset2_wide[anxiety_cols].sum(axis=1, min_count=7) * 2
)
dataset2_wide["dass21_depression_score"] = (
    dataset2_wide[depression_cols].sum(axis=1, min_count=7) * 2
)

# calculate z-scores
dataset2_wide["stress_z_score"] = (
    dataset2_wide["dass21_stress_score"]
    - dataset2_wide["dass21_stress_score"].mean()
) / dataset2_wide["dass21_stress_score"].std()
dataset2_wide["anxiety_z_score"] = (
    dataset2_wide["dass21_anxiety_score"]
    - dataset2_wide["dass21_anxiety_score"].mean()
) / dataset2_wide["dass21_anxiety_score"].std()
dataset2_wide["depression_z_score"] = (
    dataset2_wide["dass21_depression_score"]
    - dataset2_wide["dass21_depression_score"].mean()
) / dataset2_wide["dass21_depression_score"].std()

```

map parsed values to brackets to match dataset1

```

dataset2_wide["department"] = "not_recorded"
dataset2_wide["gender"] = (
    dataset2_wide["gender"].map(GENDER_FROM_RAW).fillna("other")
)

dataset2_wide["age"] = dataset2_wide["age_parsed"].map(age_int_to_bracket)
dataset2_wide["cgpa"] = dataset2_wide["cgpa_parsed"].map(cgpa_float_to_band)
dataset2_wide["academic_year"] = dataset2_wide["semester_parsed"].map(

```

```

    semester_int_to_academic_year
)

dataset2_wide["stress_level"] = dataset2_wide["dass21_stress_score"].map(
    dass21_stress_score_to_level
)
dataset2_wide["anxiety_level"] = dataset2_wide["dass21_anxiety_score"].map(
    dass21_anxiety_score_to_level
)
dataset2_wide["depression_level"] = dataset2_wide[
    "dass21_depression_score"
].map(dass21_depression_score_to_level)

for col, cats in (
    ("age", AGE_ORDER),
    ("academic_year", ACADEMIC_YEAR_ORDER),
    ("cgpa", CGPA_ORDER),
    ("stress_level", SEVERITY_ORDER),
    ("anxiety_level", SEVERITY_ORDER),
    ("depression_level", SEVERITY_ORDER),
):
    dataset2_wide[col] = pd.Categorical(
        dataset2_wide[col], categories=cats, ordered=True
    )

dataset2_wide["gender"] = pd.Categorical(
    dataset2_wide["gender"], categories=GENDER_LABELS, ordered=False
)
dataset2_wide["department"] = pd.Categorical(
    dataset2_wide["department"], categories=DEPARTMENT_CODES, ordered=False
)

dataset2_wide.info()

dataset2_clean = dataset2_wide[DATASET2_OUTPUT_COLUMNS].copy()
dataset2_clean.info()

```

merge datasets and save outputs

```

dataset1_clean.to_csv(DATA / "dataset1_clean.csv", index=False)
dataset2_wide.to_csv(DATA / "dataset2_intermediate.csv", index=False)
dataset2_clean.to_csv(DATA / "dataset2_clean.csv", index=False)

dataset1_merge = dataset1_clean[OUTPUT_COLUMNS].copy()
dataset2_merge = dataset2_clean[OUTPUT_COLUMNS].copy()

dataset1_merge["source_dataset"] = "dataset1"
dataset2_merge["source_dataset"] = "dataset2"

# ensure "source_dataset" is categorical so that its consistent with the other cols
SOURCE_DATASET_CATEGORIES = ["dataset1", "dataset2"]
dataset1_merge["source_dataset"] = pd.Categorical(
    dataset1_merge["source_dataset"], categories=SOURCE_DATASET_CATEGORIES
)
dataset2_merge["source_dataset"] = pd.Categorical(

```

```
        dataset2_merge["source_dataset"], categories=SOURCE_DATASET_CATEGORIES
    )

mhp_dataset = pd.concat([dataset1_merge, dataset2_merge], ignore_index=True)
mhp_dataset.info()
mhp_dataset.head()

mhp_dataset.to_csv(DATA / "mhp_dataset.csv", index=False)
# save to .parquet to maintain the cols dtype
mhp_dataset.to_parquet(DATA / "mhp_dataset.parquet")
```